A. Supreeth Reddy
2303A51392
Batch 06
AIAC Lab 7.3

Task 1: Fixing Syntax Errors

Scenario
You are reviewing a Python program where a basic function definition contains a syntax error.

```python
def add(a, b)
    return a + b
```

Requirements
- Provide a Python function add(a, b) with a **missing colon**
- Use an AI tool to detect the syntax error
- Allow AI to correct the function definition
- Observe how AI explains the syntax issue

Expected Output
- Corrected function with proper syntax
- Syntax error resolved successfully
- AI-generated explanation of the fix

Code

```python
def add(a, b)
    return a + b
```

```
  File "/tmp/ipython-input-1607831707.py", line 1
    def add(a, b)
                 ^
SyntaxError: expected ':'
```

Next steps: Explain error

[10] ✓ 0s

```python
def add(a, b):
    return a + b
```

AI-generated explanation of the fix

The error 'SyntaxError: expected ':' means that you are missing a colon at the end of the def add(a, b) line. Python requires a colon to mark the start

of a function's code block. I will fix this by adding a colon to the function definition.

The code def add(a, b): return a + b was executed successfully. This action defines the add function, making it available for use in subsequent cells. As this cell only defines the function and does not call it or print anything, there is no direct output.

Task 2: Debugging Logic Errors in Loops

Scenario
You are debugging a loop that runs infinitely due to a logical mistake.

```python
def count_down(n):
    while n >= 0:
        print(n)
        n += 1  # Should be n -= 1
```

Requirements
- Provide a loop with an **increment or decrement error**
- Use AI to identify the cause of infinite iteration
- Let AI fix the loop logic
- Analyze the corrected loop behavior

Expected Output
- Infinite loop issue resolved
- Correct increment/decrement logic applied
- AI explanation of the logic error

Code

```python
def count_down(n):
    while n >= 0:
        print(n)
        n += 1
```

```python
def count_down(n):
    while n >= 0:
        print(n)
        n -= 1
```

AI explanation of the logic error

The program runs but does not terminate because the loop variable is incremented instead of decremented.

This causes an infinite loop, which is a logic error and not detected by the Python interpreter.

Since the loop condition n >= 0 always remains true, the stopping condition is never reached.

This demonstrates how logical mistakes can lead to incorrect program behavior even when no syntax or runtime errors are reported.

Task 3: Handling Runtime Errors (Division by Zero)

Scenario

A Python function crashes during execution due to a division by zero error.

```python
# Debug the following code
def divide(a, b):
    return a / b

print(divide(10, 0))
```

Requirements

- Provide a function that performs division without validation
- Use AI to identify the runtime error
- Let AI add try-except blocks for safe execution
- Review AI's error-handling approach

Expected Output

- Function executes safely without crashing
- Division by zero handled using try-except
- Clear AI-generated explanation of runtime error handling

Code

```
def divide(a, b):
+    if b == 0:
+        print("Error: Cannot divide by zero!")
+        return None
     return a / b

print(divide(10, 0))
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
/tmp/ipython-input-267180578.py in <cell line: 0>()
      2     return a / b
      3
----> 4 print(divide(10, 0))

/tmp/ipython-input-267180578.py in divide(a, b)
      1 def divide(a, b):
----> 2     return a / b
      3
      4 print(divide(10, 0))

ZeroDivisionError: division by zero
```

Next steps: Explain error

[8]
✓ 0s

```
def divide(a, b):
    if b == 0:
        print("Error: Cannot divide by zero!")
        return None
    return a / b

print(divide(10, 0))
```

```
Error: Cannot divide by zero!
None
```

Clear AI-generated explanation of runtime error handling

You're encountering a ZeroDivisionError because the code is trying to divide 10 by 0. Division by zero is not allowed in mathematics or programming. I'll modify the divide function to handle this case by checking if the divisor is zero before performing the division.

The code in cell lwYSZTOy5Evf executed successfully. Your divide function now correctly handles division by zero by printing "Error: Cannot divide by zero!" and returning None, which is then printed to the output.

Task 4: Debugging Class Definition Errors

Scenario

You are given a faulty Python class where the constructor is incorrectly defined.

```python
class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

Requirements

- Provide a class definition with **missing self-parameter**
- Use AI to identify the issue in the __init__() method
- Allow AI to correct the class definition
- Understand why self is required

Expected Output

- Corrected __init__() method
- Proper use of self in class definition
- AI explanation of object-oriented error

Code

```python
class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

```
[12]
✓ 0s
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

AI explanation of object-oriented error

The error occurs because the __init__() method is defined without the self parameter.

In Python, self refers to the current object and is automatically passed to instance methods.

Without self, instance variables cannot be accessed or assigned, leading to incorrect object initialization.

Adding self to the constructor allows the class to properly store and access object attributes.

Task 5: Resolving Index Errors in Lists

Scenario

A program crashes when accessing an invalid index in a list.

```python
numbers = [1, 2, 3]
print(numbers[5])
```

Requirements

- Provide code that accesses an **out-of-range list index**
- Use AI to identify the Index Error
- Let AI suggest safe access methods
- Apply bounds checking or exception handling

Expected Output

- Index error resolved
- Safe list access logic implemented
- AI suggestion using length checks or exception handling

Code

```python
numbers = [1, 2, 3]
print(numbers[5])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-1537628302.py in <cell line: 0>()
      1 numbers = [1, 2, 3]
----> 2 print(numbers[5])

IndexError: list index out of range
```

Next steps: Explain error

Code Using Bounds Checking

[15]
0s

```python
numbers = [1, 2, 3]
index = 5

if index < len(numbers):
    print(numbers[index])
else:
    print("Error: Index out of range")
```

```
Error: Index out of range
```

Explanation

The error occurs when the program attempts to access a list index that is outside the valid range.

Bounds checking ensures that the index value is within the list length before accessing the element.

By verifying the index using len(), the program prevents an IndexError at runtime.

This approach ensures safe list access and improves the reliability of the program.